

Patch Embedding Arithmetic.

A Vision Transformer never sees a pixel grid — it sees a short sequence of flattened, linearly projected patches. This topic counts those patches exactly, sizes every matrix, and computes the attention budget. Every number on the following pages is multiplied out in full — nothing summarised or deferred.

THE EQUATION

$$N = \frac{H}{P} \cdot \frac{W}{P}, \quad z_0 = [x_{\text{cls}}; x_p^1 E; \dots; x_p^N E] + E_{\text{pos}}$$

Worked example. A $224 \times 224 \times 3$ image with $P = 16$: $N = 196$ patches, flattened patch dimension 768, projection to $d = 768$, 197 tokens to the transformer.

Topic 1 of 2 in this module · companion to the course page · v1.0

Contents

1	The claim: an image becomes a short sequence	2
2	A diagram of the embedding pipeline	3
3	The derivation: every number for 224×224 , $P = 16$	3
4	Properties and consequences	4
5	Where this sits in the track	5
A	Reproducible (NumPy)	5

1 The claim: an image becomes a short sequence

A standard Vision Transformer (ViT) does not feed pixels to an attention stack. It first cuts the $H \times W \times C$ image into a grid of non-overlapping $P \times P$ patches, flattens each patch into a vector, and projects that vector to the model width d . The result is a sequence of N token vectors, exactly the format a Transformer expects.

$$N = \frac{H}{P} \cdot \frac{W}{P}, \quad z_0 = [x_{\text{cls}}; x_p^1 E; \dots; x_p^N E] + E_{\text{pos}} \quad (1)$$

Here $x_p^i \in \mathbb{R}^{P^2 C}$ is the i -th flattened patch, $E \in \mathbb{R}^{(P^2 C) \times d}$ is the shared patch-projection matrix, $x_{\text{cls}} \in \mathbb{R}^d$ is a learnable *class* token prepended to the sequence, and $E_{\text{pos}} \in \mathbb{R}^{(N+1) \times d}$ is a learnable positional embedding added to every token. The sequence length seen by attention is $N + 1$.

Symbol	Shape	Meaning
H, W, C	scalars	image height, width, channels (e.g. 224, 224, 3)
P	scalar	patch side (e.g. 16); H, W must be divisible by P
N	scalar	number of patches = $(H/P)(W/P)$
x_p^i	$P^2 C$	one flattened patch (a row vector)
E	$(P^2 C) \times d$	shared linear projection to model width d
x_{cls}	d	learnable class token, prepended
E_{pos}	$(N+1) \times d$	learnable positional embedding, added
z_0	$(N+1) \times d$	token matrix fed to the first Transformer block

Think of patchifying as a single stride- P , kernel- P convolution with d output channels: it reads each $P \times P \times C$ tile exactly once, with no overlap, and emits one d -vector per tile. The “sequence” is just those tile-vectors read in raster (row-major) order. The image’s two-dimensional layout is then thrown away — attention sees an unordered set, which is precisely why the positional embedding E_{pos} must be added back.

2 A diagram of the embedding pipeline



The width stays at $d = 768$ from the flatten step onward because, for ViT-Base, $P^2C = 768$ happens to equal d ; in general the projection E changes the width from P^2C to d .

3 The derivation: every number for 224×224 , $P = 16$

Step 1 — patch count. With $H = W = 224$ and $P = 16$,

$$\frac{H}{P} = \frac{224}{16} = 14, \quad N = \frac{H}{P} \cdot \frac{W}{P} = 14 \times 14 = 196.$$

Step 2 — flattened patch dimension. Each patch holds $P \times P \times C$ scalars:

$$P^2C = 16 \times 16 \times 3 = 256 \times 3 = 768.$$

Step 3 — projection matrix. E maps $\mathbb{R}^{768} \rightarrow \mathbb{R}^{768}$, so $E \in \mathbb{R}^{768 \times 768}$, holding $768 \times 768 = 589,824 \approx 0.59\text{M}$ weights (plus a 768-vector bias).

Step 4 — token count. Prepending the class token gives

$$N + 1 = 196 + 1 = 197 \text{ tokens}, \quad z_0 \in \mathbb{R}^{197 \times 768}.$$

Step 5 — attention budget. Self-attention compares every token to every token, so the score matrix $QK^\top$ has

$$(N + 1)^2 = 197^2 = 38,809 \text{ entries.}$$

For comparison, a 512-token text sequence costs $512^2 = 262,144$ entries — about $262,144/38,809 \approx 6.75\times$ more. Vision sequences are short.

Image	P	H/P	N	tokens $N+1$	attn pairs $(N+1)^2$
224×224	16	14	196	197	38,809
224×224	32	7	49	50	2,500
384×384	16	24	576	577	332,929
384×384	32	12	144	145	21,025

Doubling the patch side P from 16 to 32 divides N by 4 and the attention cost by roughly 16; raising the resolution from 224 to 384 multiplies N by $(384/224)^2 \approx 2.94$.

Sequence length is governed by the *ratio* H/P , squared — not by the pixel count directly. ViT’s cost knob is the patch size P : bigger patches mean fewer, coarser tokens and a quadratically cheaper attention map, at the price of spatial resolution.

3.1 A patch grid as a heatmap

A 64×64 image at $P = 16$ yields a 4×4 grid of $N = 16$ patches. Below, each cell is one patch, numbered in raster order; the indigo intensity rises with the patch index to make the row-major flattening order visible. This is the order in which patches enter the sequence.

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

4 Properties and consequences

1. **Divisibility constraint.** P must divide both H and W ; otherwise the patch grid is ill-defined and implementations crop, pad, or interpolate.
2. **Linear, not lossless.** The flatten-then-project step is a single affine map. Two distinct patches can map to nearby embeddings; no nonlinearity is applied before the first block.
3. **Parameter sketch (ViT-Base, $d = 768$, $L = 12$).** The patch embedding holds $P^2 C \cdot d = 768 \times 768 = 0.59\text{M}$ weights. Each block contributes roughly $12d^2$ in its projections (four $d \times d$ attention matrices $\approx 4d^2$, plus a $4d$ MLP that adds $\approx 8d^2$): $12 \times 768^2 = 7.08\text{M}$ per layer, so $12 \times 7.08\text{M} = 84.9\text{M}$ across the stack — the bulk of ViT-Base’s $\sim 86\text{M}$ parameters lives in the blocks, not the embedding.
4. **Positional embedding is learned and 1D.** Although patches come from a 2D grid, the default ViT adds a learnable length- $(N+1)$ table indexed by sequence position, with no built-in 2D structure; the model must learn any spatial relationships.

The positional embedding is tied to a fixed N . Change the input resolution and N changes, so the pretrained E_{pos} no longer has the right length. Fine-tuning at higher resolution requires *interpolating* the positional table (reshaping it to the 2D grid, resizing, and flattening back) — you cannot simply reuse the 197-row table for a 577-token sequence.

5 Where this sits in the track

Link	Relevance
Module 0.1	Scaled dot-product attention — the $(N+1)^2$ score matrix counted here.
Module 0.x	Positional encodings — here they are <i>learned</i> , not sinusoidal.
Topic 2 (next)	Why ViT lacks the inductive bias a CNN gets for free, and DeiT's distillation fix.

A Reproducible (NumPy)

Inputs: image $H=W=224$, $C=3$, patch $P=16$, width $d=768$. Derived: $N=196$, $P^2C=768$, tokens $=197$, $E \in \mathbb{R}^{768 \times 768}$ (589,824 weights), attention pairs $197^2=38,809$.

```
import numpy as np
H = W = 224; C = 3; P = 16; d = 768
N = (H // P) * (W // P)          # 196
flat = P * P * C                 # 768
tokens = N + 1                   # 197
E_params = flat * d              # 589824
attn_pairs = tokens ** 2         # 38809
print(N, flat, tokens, E_params, attn_pairs)
# -> 196 768 197 589824 38809

for (Hh, Pp) in [(224,16),(224,32),(384,16),(384,32)]:
    Nn = (Hh // Pp) ** 2
    print(Hh, Pp, Nn, Nn + 1, (Nn + 1) ** 2)
# -> 224 16 196 197 38809
# -> 224 32 49 50 2500
# -> 384 16 576 577 332929
# -> 384 32 144 145 21025

per_layer = 12 * d * d          # 7077888
stack = 12 * per_layer         # 84934656
print(per_layer, stack)        # 7077888 84934656
```